

Procedurally Animated Inverse Kinematics for n-DOF Linkages

Finian O'Neal

<https://finianoneal.cv>

finianoneal@gmail.com

Notation

Symbol	Description
x_D	Target position
x_E	End effector position
x_o	Linkage origin
φ	angular configuration
l	Link lengths
α	Step weight
β	Step decay
γ	Penalty weight
f	Forward kinematics
ε	Position error
P	Penalty function
G	Error function
v	angular velocity

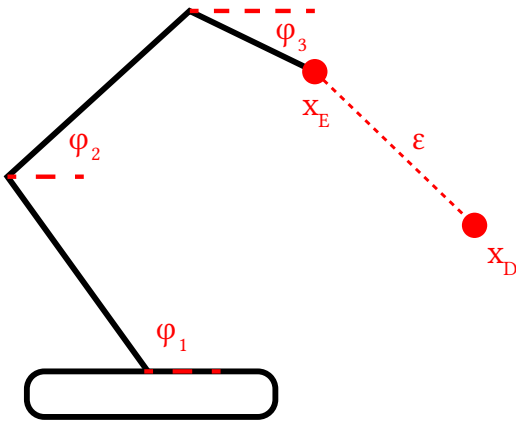


Figure 1: Example arm with $n = 3$

Motivation

When coming up with concepts for my portfolio website, I became fixated on the idea of including some sort of engineering-related interactive element visitors could play with. My thought was that this element could help differentiate my site while additionally giving me a chance to walk visitors through my problem-solving process. After some deliberation, I chose to design a simulated robot arm which could be controlled by site visitors.

I chose this particular project for a few reasons:

1. The result should be easy to recognize and intuitive to a wide audience
2. The underlying problem is interesting with many possible approaches and applications in multiple fields
3. The underlying problem is new to me

The problem in controlling a robot arm (or any such linkage) based on where you want it to go is that, for all interesting scenarios, there may be multiple solutions or no solution at all. If you hold a pen tip stationary on paper while moving your arm about or try to reach something too far away you will see what I mean. This is the problem of Inverse Kinematics (IK): for an arbitrary linkage composed of $n > 1$ rotatable, rigid links, any function F which maps target position x_D to solution joint angle configuration φ_D will be under-constrained. Your brain handles this problem subconsciously when moving your limbs, but designing an algorithm which finds not just a solution but an *ideal* solution is not straightforward. The rest of this

paper will summarize my work on this problem in the context of designing a controller for a 2D robot arm simulation. While the end solution was written in JavaScript for integration into my website, the code itself is not particularly interesting and has been omitted for brevity.

Approach

Solution Criteria

1. **Nearby solutions:** The controller should prefer solutions which involve minimal joint movement.
2. **Animation:** The arm should smoothly rotate between solutions.
3. **Reasonability:** When no solutions exist, the arm should extend in the direction of the target.
4. **Generality:** The controller should function for 2D linkages of any length.

Method Selection

Both analytical and numerical methods exist for IK, however analytical methods can be eliminated immediately as they are not generalizable [1]. There are two broad categories of methods for tackling IK numerically. Rootfinding methods employ algorithms such as the Newton-Raphson, Secant, or Bracketing methods to solve for the joint angles at which the position error is zero [1,2]. Minimization methods, on the other hand, focus on minimizing the position error. While the two sound similar, the key distinction is that minimization methods tend to be slower but yield better results when there is no solution (i.e there are no roots to find) [1]. Additionally, the function being minimized can be chosen such that the controller is incentivized to find nearby solutions. Given the criteria, this is the option I chose.

The last remaining question is what minimization method to employ. While sophisticated options capable of handling complex physical constraints exist [3], they are unnecessary for this project. I chose to use Gradient descent, which is a straightforward iterative approach.

Implementation

Let the position error $\varepsilon = \|x_D - x_E\|_2$, where x_D is the target position and x_E is the current end effector position. The value of x_E can be found by solving the inverse problem to IK: forward kinematics. Luckily this is a much easier problem to tackle. Given a 2D linkage of n links, with configuration vector $\varphi_D \in \mathbb{R}^{n \times 1}$, lengths vector $l \in \mathbb{R}^{n \times 1}$, and linkage origin $x_o \in \mathbb{R}^{2 \times 1}$ trigonometry shows the solution to be

$$x_E \equiv f(\varphi) = \sum_{i=1}^n \left[l_i \begin{pmatrix} -\sin(\varphi_i) \\ \cos(\varphi_i) \end{pmatrix} \right] + x_o$$

Thus, ε can be rewritten in terms of $f(\varphi)$ to create the most basic error function for IK: $\varepsilon(\varphi) = \|x_D - f(\varphi)\|_2$. From vector calculus, the gradient of the error $\nabla \varepsilon(\varphi) = \left(\frac{\partial \varepsilon}{\partial \varphi_1}, \frac{\partial \varepsilon}{\partial \varphi_2}, \dots, \frac{\partial \varepsilon}{\partial \varphi_n} \right)$ shows the configuration space direction in which the error increases the most. The idea of gradient descent is to take a step in the opposite direction and re-evaluate until the error is minimized. Once a solution is found the linkage can move from its current angular configuration to the new one.

```

1 let  $\varphi^0$  = random initial angular config.
2 let  $N_{\max}$  = iter. limit
3 let  $\alpha$  = fixed step size
4 let  $\varepsilon_{tol}$  = error tolerance
5 func grad_descent:
6   for  $i \in 0, 1, 2, \dots, (N_{\max} - 1)$ 
7      $\varphi^{i+1} = \varphi^i - \alpha \nabla \varepsilon(\varphi^i)$ 
8     if  $|\varepsilon| \leq \varepsilon_{tol}$  return  $\varphi^{i+1}$ 
9   return  $\varphi^{i+1}$ 
```

Figure 2: My initial algorithm for gradient descent

Iterations

While this initial algorithm technically worked, it did not meet all criteria and frequently failed to converge to a solution. Through iteration, a couple of improvements were made:

1. Adaptive Step Size

Using a fixed step size α can slow or prevent convergence. If α is too large, as shown in Figure 2, the algorithm may drastically overshoot the minimum and increase error. However, if α is too small the algorithm may not converge fast enough or may get stuck in local minimums [1].

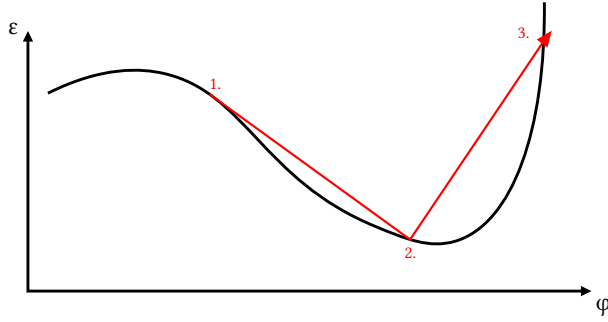


Figure 3: Large step size overshoots min. and may oscillate without convergence

One solution is to decrease step size with each iteration. This allows the algorithm to take large steps initially, which increases convergence speed, while mitigating overshoot. I found sufficient success by tuning $\alpha = \alpha_0 e^{-\beta i}$, where α_0 is the initial step size and β controls the rate of decay.

2. Penalty Function

Criteria 1 states that the controller should prefer nearby solutions. In its initial state there was no reason for the algorithm to differentiate between minimums. This lead to unnatural movements as far-away solutions caused excessive joint rotation to get between targets. My first improvement was to “warm-start” the algorithm by setting φ^0 to the current angular configuration. While this increased the likelihood of finding a nearby solution, it was not sufficient. A different approach is to change the function being minimized. Let penalty function $P(\varphi) = \|\varphi - \varphi^0\|_2$. By incorporating $\varepsilon(\varphi)$ and $P(\varphi)$ into a new error function

$$G(\varphi) = \varepsilon(\varphi) + \gamma P(\varphi)$$

the shape of the error is changed such that points corresponding to far-away configurations are artificially raised up, encouraging nearby solutions.

The scaling factor γ is necessary for tuning, as if the contribution for $P(\varphi)$ is too large the controller will prioritize finding nearby configurations over finding solutions.

3. Second Order Control

The second criteria is that the arm must smoothly rotate between solutions. The easiest solution would be to linearly interpolate between configurations over a set time interval, however this would create a period of unresponsiveness. An alternative approach would be to let solutions dictate the dynamics through procedural animation. Let φ^{old} be the previous target and let φ be the angular configuration at the current time step. Let angular acceleration $\ddot{\varphi} = \varphi - \varphi^{old}$. By numerical integration the same result as interpolation can be achieved however the simulation will be able to accommodate new targets while in motion. One step further is to feed the angular velocity $\dot{\varphi}$ into a second order system $m\ddot{\varphi} + c\dot{\varphi} + k\varphi = v$. By tuning of the parameters m, c, k different behaviors can be achieved. The current parameters result in a slightly under-damped system, which allows the controller to reach new targets quickly while giving the animation subtle character.

References

- [1] Hauser, Kris. “Inverse Kinematics.” Robotic Systems. Accessed April 14, 2026. <https://motion.cs.illinois.edu/RoboticSystems/InverseKinematics.html>.
- [2] Atkinson, Kendall E., and Weimin Han. Elementary Numerical Analysis, 2004.
- [3] Votroubek, Tomáš & Kroupa, Tomáš. (2024). Globally Optimal Inverse Kinematics as a Non-Convex Quadratically Constrained Quadratic Program. IEEE Robotics and Automation Letters. PP. 1-6. 10.1109/LRA.2024.3398433.